

Instituto Federal

- **Campus Goiânia**

Bacharelado em Sistemas de Informação

POO II

Prof. Dory Gonzaga Rodrigues

**Ementa****INTRODUÇÃO**

- Controle de Versão
- Git
- GitHub





Controle de Versão

O que é controle de versão?

O controle de versões é o processo que visa gerenciar qualquer tipo de documento, durante todo o ciclo de vida, desde a sua criação, alterações e exclusão, de forma eficiente, organizada e prática.

Este tipo de sistema é comumente utilizado em projetos onde vários integrantes contribuem paralelamente com o seu desenvolvimento.

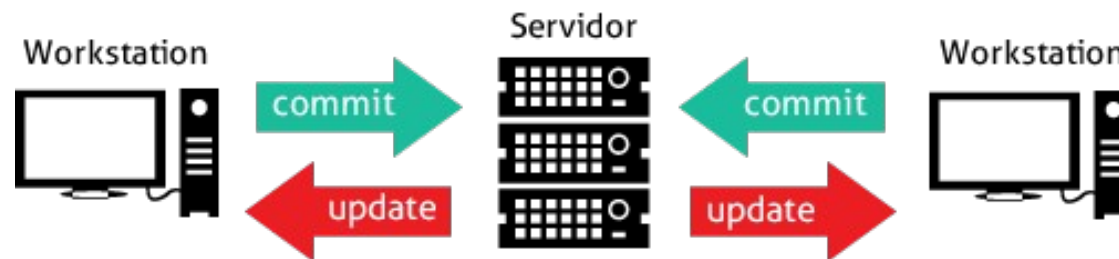
Em projetos de softwares, o controle de versões pode ser utilizado para controlar o histórico de desenvolvimento dos códigos-fontes e também da documentação dos mesmos, permite que vários programadores trabalhem no mesmo projeto, e permite um comparativo entre várias versões do projeto.



Controle de Versão

Como funciona?

Existe um servidor denominado de Repositório Remoto. Nele fica o projeto (código-fonte) com todo o histórico (modificações) de cada versão. O desenvolvedor pode baixar a última versão, realizar modificações e posteriormente atualizar a versão contida no servidor.



Essa sincronização de arquivos entre a estação de trabalho e o servidor é feito através dos comandos ***commit*** e ***update***.

commit envia os arquivos alterados para o Repositório, gerando um novo histórico de atualização.

update recebe a última versão do projeto na estação de trabalho.



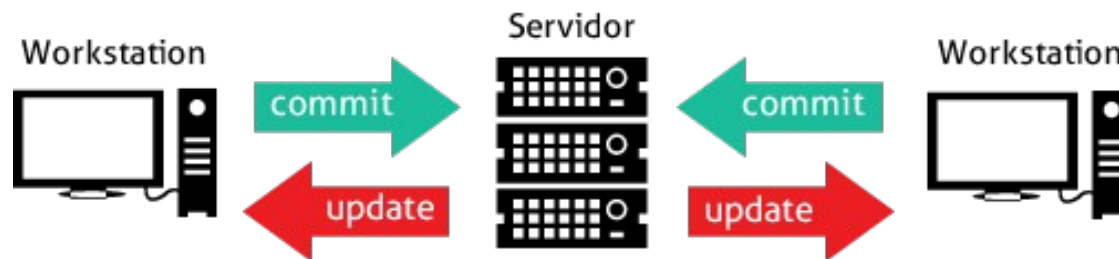
Controle de Versão

Topologia (Estrutura)

Controle de Versão Centralizado

Repositório Único (Servidor Central) e várias cópias do projeto nas diversas estações de trabalho.

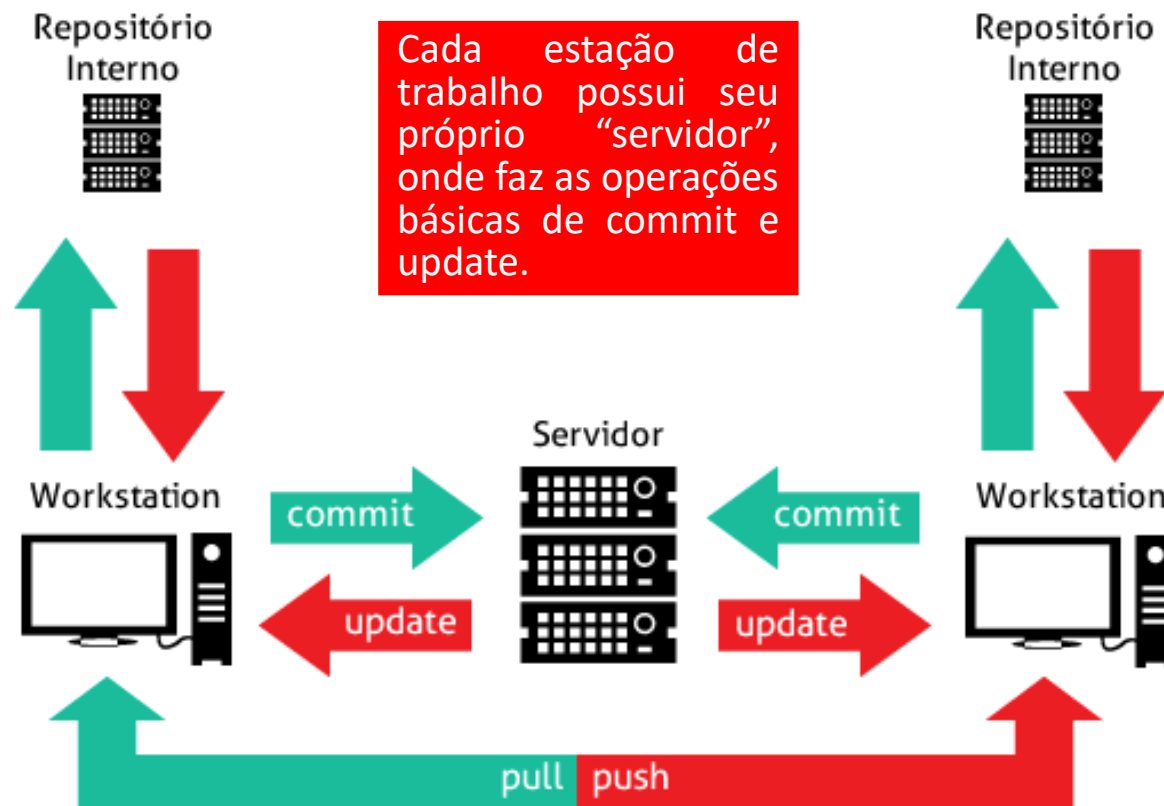
A comunicação entre as estações de trabalho necessariamente passam pelo servidor central.



Controle de Versão

Topologia (Estrutura)

Controle de Versão Distribuído



cada estação de trabalho pode comunicar-se diretamente com a outra

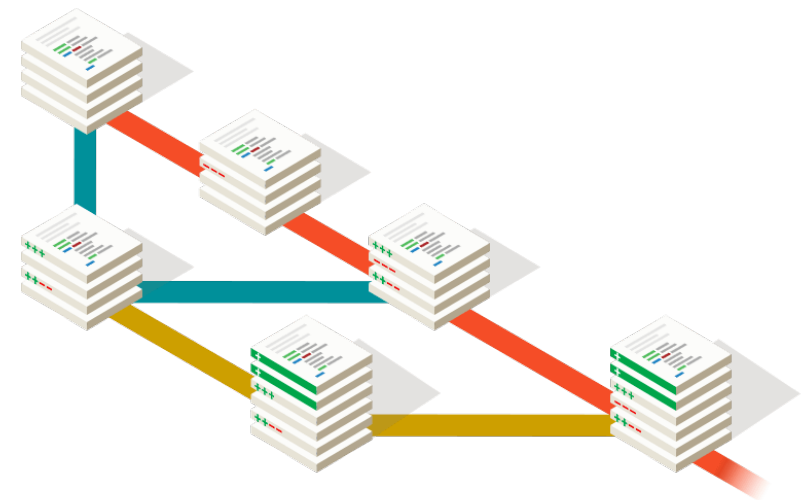


Git

Git

Git é um sistema de controle de versão distribuído gratuito e de código aberto projetado para lidar com tudo, desde projetos pequenos a muito grandes com velocidade e eficiência.

É rápido, possui um design simples, fácil instalação, configuração e baixa curva de aprendizado.



Site: <https://git-scm.com/>

Git

Características do Git

Histórico do documento

Qualquer alteração no projeto, desde uma simples palavra, ele irá controlar. Isso ajuda muito para saber as alterações de uma versão para outra.

Branches (Ramificações)

Permite que você crie versões diferentes do mesmo sistema sem afetar outros.

Distribuído

Cada usuário basicamente terá um backup completo do servidor principal.

Operações locais

Operações quase que instantâneas, pois em sua maioria são realizadas localmente e não dependem de conexão com o servidor.



Características do Git

Snapshots

São uma espécie de foto dos arquivos num determinado “momento”.

Cada *commit* que o usuário faz no seu diretório Git, é como se fosse feita uma captura de todos os arquivos presentes no diretório e criada uma referência para essas capturas.

Integridade

A integridade das informações de cada projeto são asseguradas por um *checksum* (chave de verificação de integridade). Com o *checksum*, o Git assegura que o usuário não irá perder informação em movimentação ou corromper arquivos sem que ele mesmo perceba. Para realizar *checksum*, o Git usa uma implementação de *hash* SHA-1, calculada a partir de um arquivo ou estrutura do diretório Git.



Características do Git

Estados de operação

Quando se está atuando com um diretório Git, o mesmo obriga os arquivos a estarem em um dos 3 estados de operação:

Consolidado (***committed***): o repositório Git é onde se encontram todos os arquivos em sua forma *committed*

Modificado (***modified***): arquivos modificados e, no momento, se encontram diferentes do que está *committed* no diretório Git.

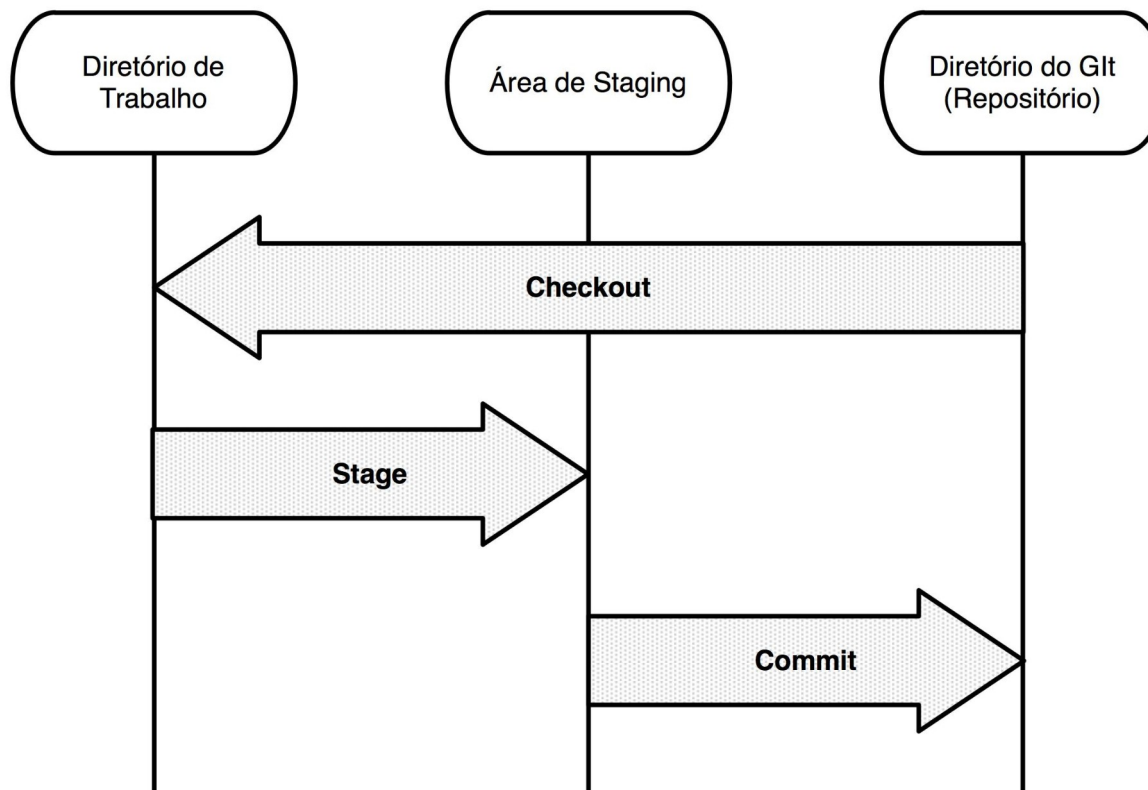
Preparado (***staged***): arquivos enviados pra área de *staged* para que as modificações efetuadas sejam registradas como *committed*.

Para manter a consistência desses estados, o Git divide seu diretório em três seções que são fundamentais para o projeto como um todo: o repositório Git, o diretório de trabalho e a área de preparação.



Fluxo de trabalho do Git

O fluxo de trabalho no Git funciona basicamente da seguinte maneira:



1 - os arquivos são modificados no diretório de trabalho;

2 - o usuário seleciona os arquivos que pretende consolidar no repositório Git e os envia para a área de preparação

3 - executa um *commit*, o qual envia os arquivos presentes na área de preparação para o repositório Git.

**Git**

GitHub

É um serviço de hospedagem de repositórios Git remotos.

Possui uma interface gráfica web: <https://github.com/>

É uma plataforma social
(usuários, página de perfil, seguidores, colaboração, etc.)

Maior serviço do mundo de hospedagem de projetos de código aberto

Gratuito para projetos de código aberto, pago para projetos privados

Alternativas: BitBucket, GitLab, etc

Git

Instalação do Git

Passo 1

Baixe o instalador do site oficial <https://git-scm.com/downloads>

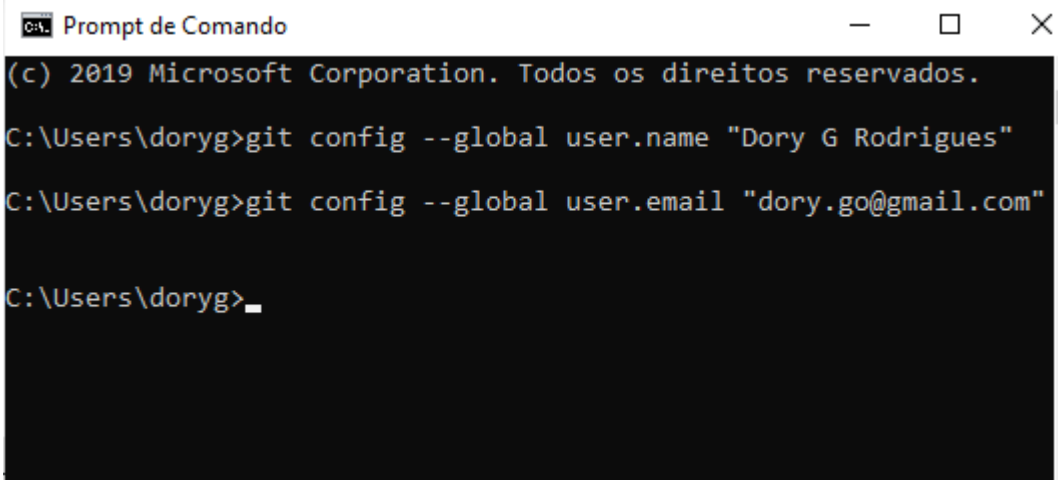
Passo 2

Depois de baixado, clique duas vezes no arquivo para iniciar o assistente de instalação. Basta seguir as instruções na tela, clicando em *Next*. Ao término, clique em *Finish* para concluir com êxito a instalação.

Passo 3

Abra o *prompt* de comando e digite os seguintes comandos no terminal:

```
git config --global user.name "nome"  
git config --global user.email "email"
```



```
Prompt de Comando  
(c) 2019 Microsoft Corporation. Todos os direitos reservados.  
C:\Users\doryg>git config --global user.name "Dory G Rodrigues"  
C:\Users\doryg>git config --global user.email "dory.go@gmail.com"  
C:\Users\doryg>
```



Git

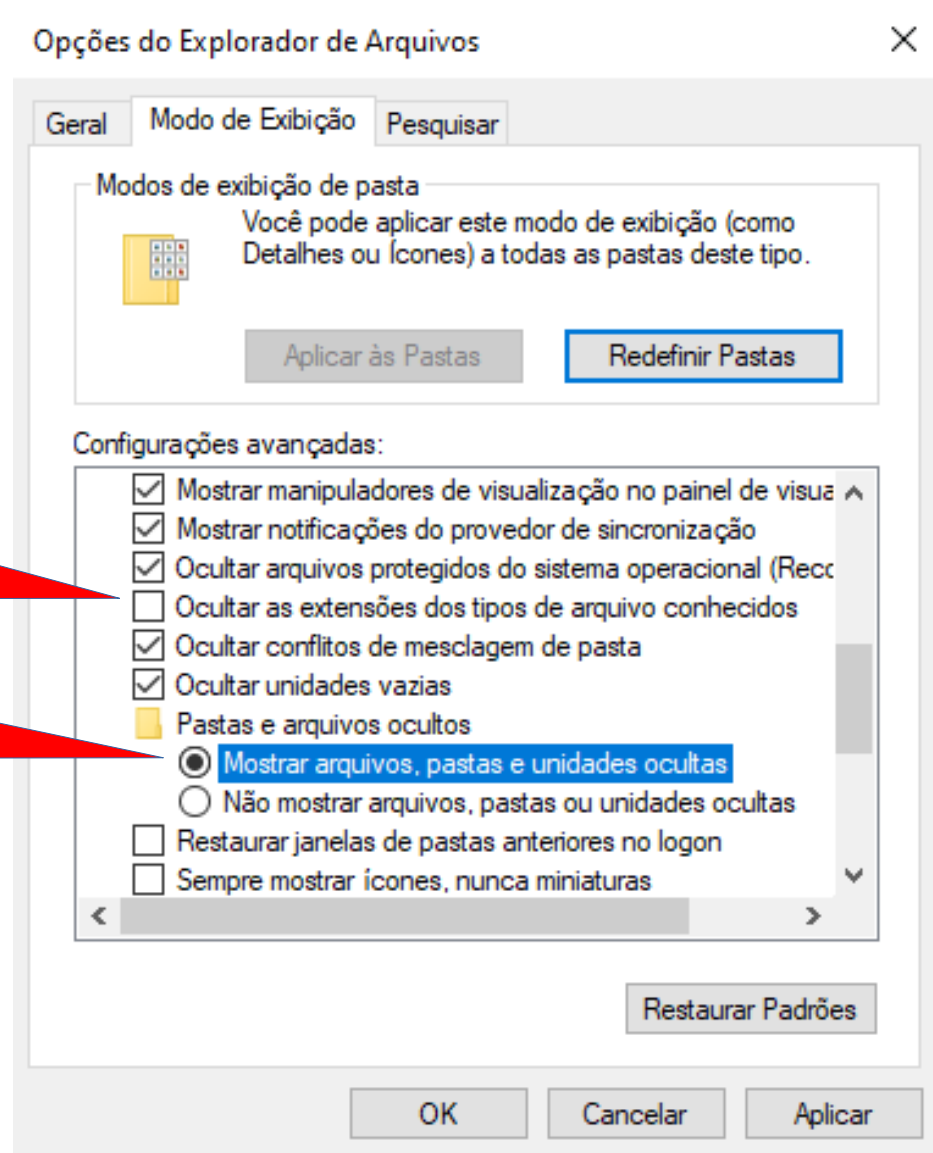
Instalação do Git

Passo 4

No Windows, configure o sistema para mostrar extensões de arquivos conhecidos.

Desmarque
esta opção

Marque
esta opção





Git

Usando o Git com GitHub

Passo 1

a - Acesse <https://github.com/>

b - Crie uma conta

c – Entre

The screenshot shows the GitHub homepage with a dark background. On the left, the text "Built for developers" is prominently displayed in white. Below it, a paragraph describes GitHub as a development platform. On the right, there is a white sign-up form with three input fields: "Username", "Email", and "Password". Below the "Password" field, there is a note about password requirements and a "Learn more" link. A large green button labeled "Sign up for GitHub" is positioned below the form. At the bottom of the form, there is a disclaimer about agreeing to the Terms of Service and Privacy Statement.

Why GitHub? Team Enterprise Explore Marketplace Pricing Search GitHub Sign in Sign up

Built for developers

GitHub is a development platform inspired by the way you work. From open source to business, you can host and review code, manage projects, and build software alongside 50 million developers.

Username

Email

Password

Make sure it's at least 15 characters OR at least 8 characters including a number and a lowercase letter. [Learn more.](#)

Sign up for GitHub

By clicking "Sign up for GitHub", you agree to our [Terms of Service](#) and [Privacy Statement](#). We'll occasionally send you account related emails.

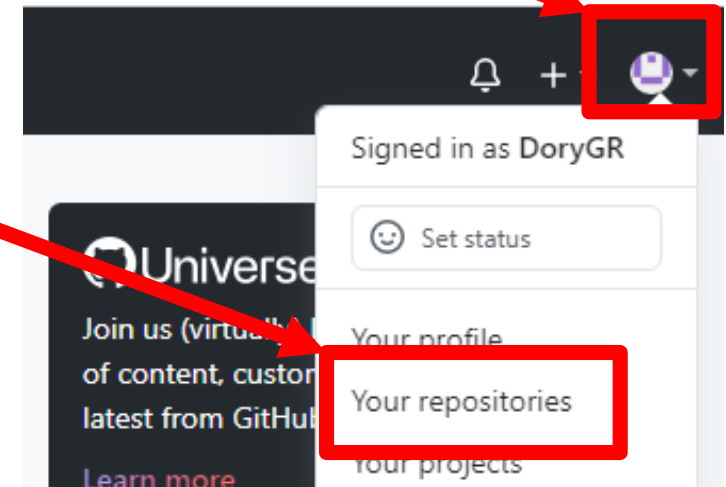


Git

Usando o Git com GitHub

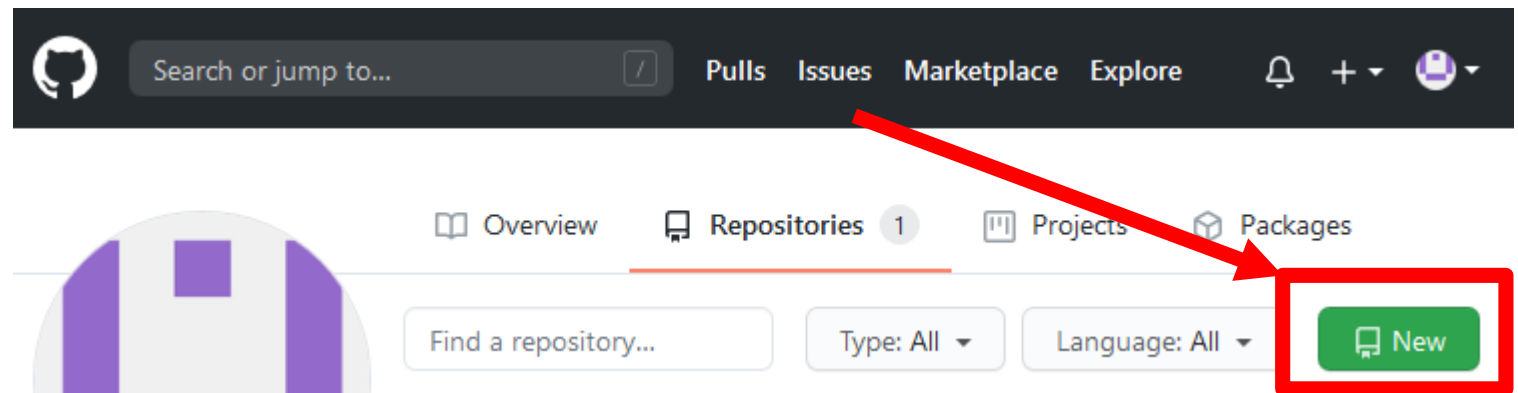
Passo 2

No canto superior direito, clique no ícone e escolha “Your repositories”



Passo 3

Clique no botão New





Git

Usando o Git com GitHub

Passo 4

Preencha o formulário com as seguintes informações:

Nome do repositório

Marque se será público ou privado

Marque para adicionar um arquivo `.gitignore` e escolha o *template* "Java"

Observe o nome *default* do *branch*

Clique no botão

The screenshot shows the GitHub repository creation interface. The 'Owner' is 'DoryGR' and the 'Repository name' is 'aulajpa'. The 'Description' is 'Exemplo do uso de JPA'. The repository is set to 'Private'. Under 'Initialize this repository with:', the 'Add .gitignore' option is checked, and the 'Choose a license' option is unchecked. The 'main' branch is selected as the default branch. The 'Create repository' button is at the bottom.

Owner * DoryGR / Repository name * aulajpa ✓

Great repository names are short and memorable. Need inspiration? How about `fictional-potato`?

Description (optional)
Exemplo do uso de JPA

☐ Public
Anyone on the internet can see this repository. You choose who can commit.

☒ Private
You choose who can see and commit to this repository.

Initialize this repository with:
Skip this step if you're importing an existing repository.

☐ Add a README file
This is where you can write a long description for your project. [Learn more.](#)

☒ Add .gitignore
Choose which files not to track from a list of templates. [Learn more.](#)
.gitignore template: Java

☐ Choose a license
A license tells others what they can and can't do with your code. [Learn more.](#)

This will set main as the default branch. Change the default name in your [settings](#).

Create repository



Git

Usando o Git com GitHub

Passo 5

Nome do Repositório criado

The screenshot shows the GitHub interface for a newly created repository named 'aulajpa' under the user 'DoryGR'. The repository is private. The main branch is selected. The commit history shows an initial commit by 'DoryGR' 4 minutes ago. The repository name 'DoryGR / aulajpa' is highlighted with a red box and a red arrow pointing to it from the text 'Nome do Repositório criado'.

Repository: DoryGR / aulajpa (Private)

main 1 branch 0 tags

Go to file Add file Code

DoryGR Initial commit a5b9957 4 minutes ago 1 commits

.gitignore Initial commit 4 minutes ago

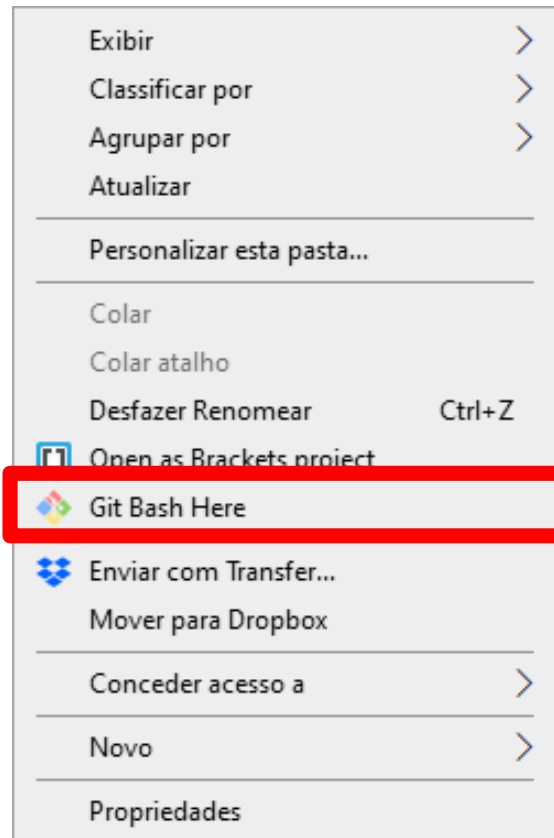
Add a README with an overview of your project. Add a README

Git

Usando o Git com GitHub

Passo 6

No seu computador, dentro da pasta do projeto Java, abra um terminal Git, clicando com o botão direito e escolhendo a opção mostrada abaixo:



Git

Usando o Git com GitHub

Passo 7

Verifique as configurações do Git

Comando git config --list

Se as configurações do nome e e-mail não estiverem criadas, execute o passo 3 da instalação do Git



```
MINGW64; c:/Users/doryg/Documents/ws-spring/aulajpa
doryg@Dory-Dell-XPS MINGW64 ~/Documents/ws-spring/aulajpa (master)
$ git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=openssl
http.sslcainfo=C:/Git/mingw64/ssl/certs/ca-bundle.crt
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
user.name=Dory G. Rodrigues
user.email=dory.go@gmail.com
core.repositoryformatversion=0
core.filemode=false
core.bare=false
core.logallrefupdates=true
core.symlinks=false
core.ignorecase=true

doryg@Dory-Dell-XPS MINGW64 ~/Documents/ws-spring/aulajpa (master)
$
```

Git

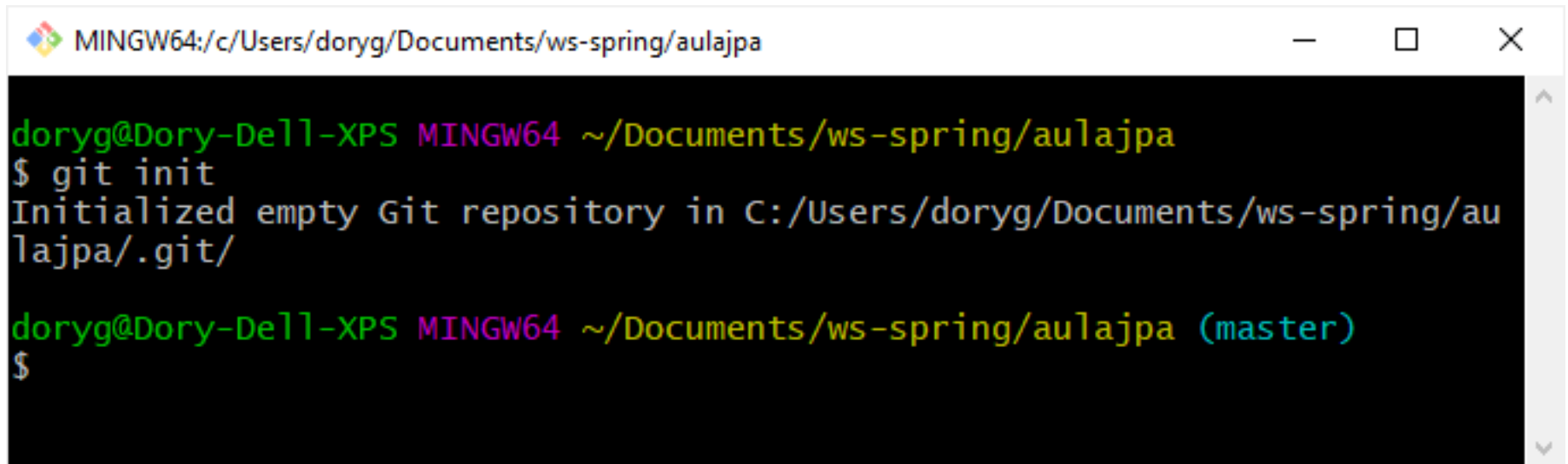
Usando o Git com GitHub

Passo 8

Crie um repositório local na pasta do seu projeto Java

Comando

`git init`



```
MINGW64:/c/Users/doryg/Documents/ws-spring/aulajpa
doryg@Dory-Dell-XPS MINGW64 ~/Documents/ws-spring/aulajpa
$ git init
Initialized empty Git repository in C:/Users/doryg/Documents/ws-spring/aulajpa/.git/
doryg@Dory-Dell-XPS MINGW64 ~/Documents/ws-spring/aulajpa (master)
$
```

Git

Usando o Git com GitHub

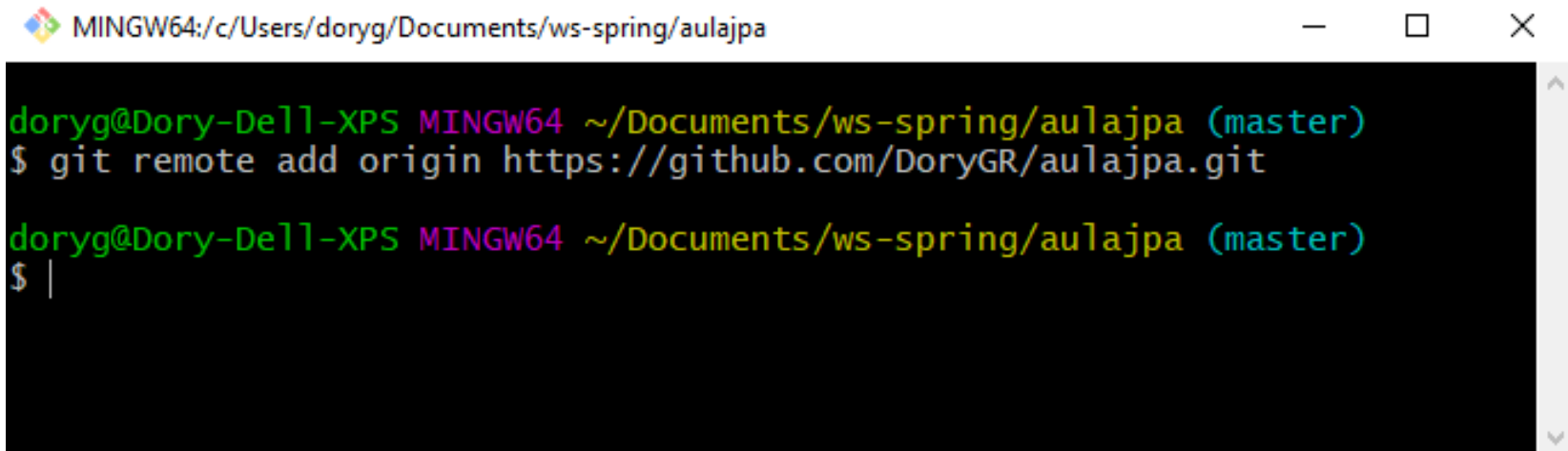
Passo 9

Associe o repositório local ao repositório remoto

Atenção: deve ser usado o **nome/repositório** criado no passo 4

Comando

`git remote add origin https://github.com/usuario/repositorio.git`



```
MINGW64:/c:/Users/doryg/Documents/ws-spring/aulajpa
doryg@Dory-Dell-XPS MINGW64 ~/Documents/ws-spring/aulajpa (master)
$ git remote add origin https://github.com/DoryGR/aulajpa.git
doryg@Dory-Dell-XPS MINGW64 ~/Documents/ws-spring/aulajpa (master)
$ |
```

Git

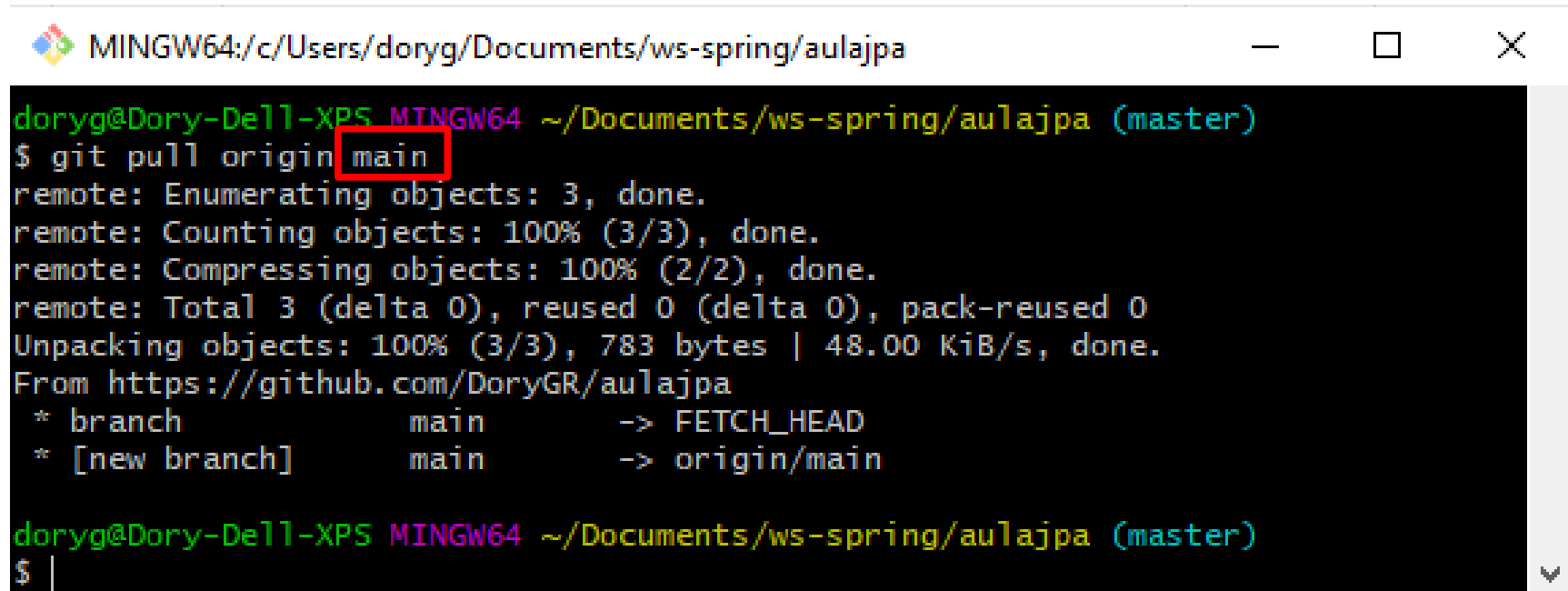
Usando o Git com GitHub

Passo 10

Atualize o repositório local com a versão disponível no repositório remoto.

Comando

`git pull origin nome_default_do_branch`



```
MINGW64:/c/Users/doryg/Documents/ws-spring/aulajpa
doryg@Dory-Dell-XPS MINGW64 ~/Documents/ws-spring/aulajpa (master)
$ git pull origin main
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (3/3), 783 bytes | 48.00 KiB/s, done.
From https://github.com/DoryGR/aulajpa
* branch          main          -> FETCH_HEAD
* [new branch]     main          -> origin/main
doryg@Dory-Dell-XPS MINGW64 ~/Documents/ws-spring/aulajpa (master)
$ |
```

Git

Usando o Git com GitHub

Passo 11

Verifique se temos arquivos, no repositório local, não rastreados. Arquivos não rastreados significa que não estão no repositório remoto.

Comando

`git status`

MINGW64:/c/Users/doryg/Documents/ws-spring/aulajpa



```
doryg@Dory-Dell-XPS MINGW64 ~/Documents/ws-spring/aulajpa (master)
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    .classpath
    .project
    .settings/
    pom.xml
    src/
    target/

nothing added to commit but untracked files present (use "git add" to track)
```


Git

Usando o Git com GitHub

Passo 12

Adicione os arquivos na área de preparação (área *Stage*) para serem enviados para o GitHub

Comando
`git add .`

Passo 13

Visualize os
arquivos prontos
para o committed

Comando
`git status`

```
MINGW64:/c/Users/doryg/Documents/ws-spring/aulajpa
doryg@Dory-Dell-XPS MINGW64 ~/Documents/ws-spring/aulajpa (master)
$ git add .

doryg@Dory-Dell-XPS MINGW64 ~/Documents/ws-spring/aulajpa (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   .classpath
    new file:   .project
    new file:   .settings/org.eclipse.jdt.apt.core.prefs
    new file:   .settings/org.eclipse.jdt.core.prefs
    new file:   .settings/org.eclipse.m2e.core.prefs
    new file:   pom.xml
    new file:   src/main/java/application/Program.java
    new file:   src/main/java/aulajpa/Aluno.java
    new file:   src/main/java/aulajpa/Curso.java
    new file:   src/main/resources/META-INF/persistence.xml
    new file:   target/classes/META-INF/MANIFEST.MF
    new file:   target/classes/META-INF/maven/br.com.ifgoias/aulajpa/pom.properties
    new file:   target/classes/META-INF/maven/br.com.ifgoias/aulajpa/pom.xml
    new file:   target/classes/META-INF/persistence.xml

doryg@Dory-Dell-XPS MINGW64 ~/Documents/ws-spring/aulajpa (master)
$
```

Git

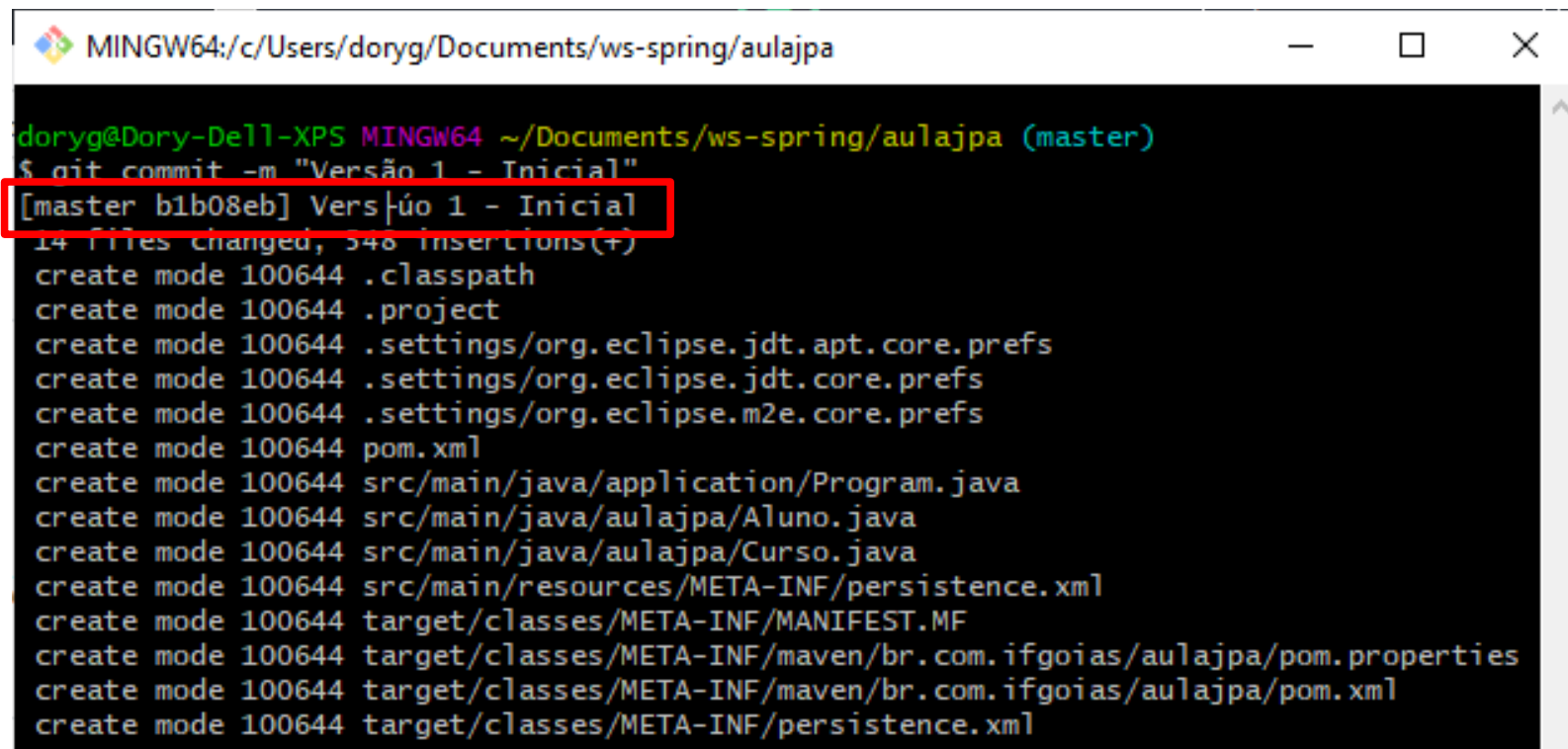
Usando o Git com GitHub

Passo 14

Cria uma nova versão do projeto no repositório local

Comando

`git commit -m "título da versão"`



```
MINGW64:/c:/Users/doryg/Documents/ws-spring/aulajpa
doryg@Dory-De11-XPS MINGW64 ~/Documents/ws-spring/aulajpa (master)
$ git commit -m "Versão 1 - Inicial"
[master b1b08eb] Versão 1 - Inicial
14 files changed, 548 insertions(+)
create mode 100644 .classpath
create mode 100644 .project
create mode 100644 .settings/org.eclipse.jdt.apt.core.prefs
create mode 100644 .settings/org.eclipse.jdt.core.prefs
create mode 100644 .settings/org.eclipse.m2e.core.prefs
create mode 100644 pom.xml
create mode 100644 src/main/java/application/Program.java
create mode 100644 src/main/java/aulajpa/Aluno.java
create mode 100644 src/main/java/aulajpa/Curso.java
create mode 100644 src/main/resources/META-INF/persistence.xml
create mode 100644 target/classes/META-INF/MANIFEST.MF
create mode 100644 target/classes/META-INF/maven/br.com.ifgoias/aulajpa/pom.properties
create mode 100644 target/classes/META-INF/maven/br.com.ifgoias/aulajpa/pom.xml
create mode 100644 target/classes/META-INF/persistence.xml
```

Git

Usando o Git com GitHub

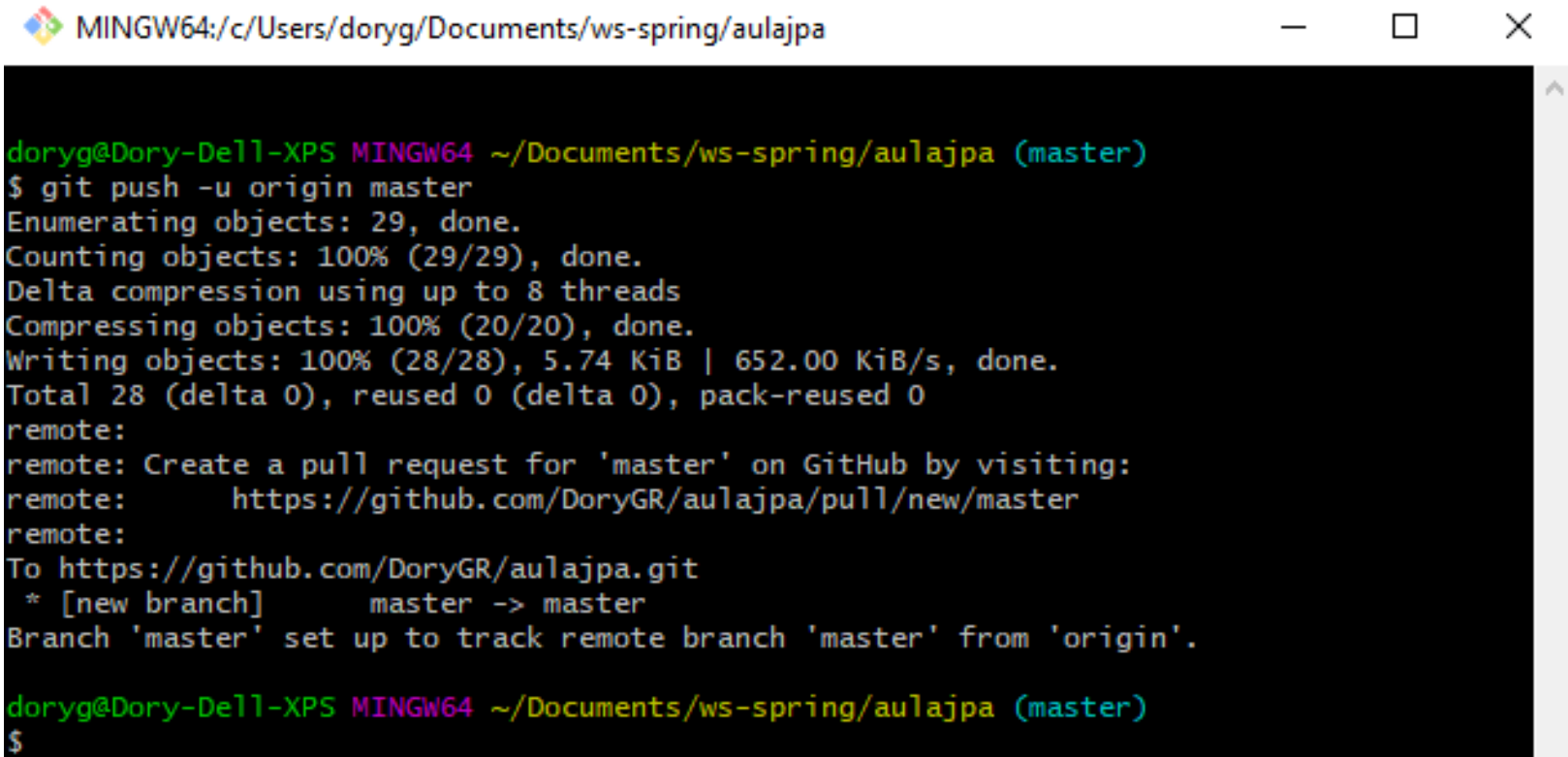
Passo 15

Envia a nova versão do projeto para o repositório remoto

Comando

`git push -u origin master`

Será solicitado o usuário e a senha do GitHub



```
MINGW64:/c/Users/doryg/Documents/ws-spring/aulajpa

doryg@Dory-Dell-XPS MINGW64 ~/Documents/ws-spring/aulajpa (master)
$ git push -u origin master
Enumerating objects: 29, done.
Counting objects: 100% (29/29), done.
Delta compression using up to 8 threads
Compressing objects: 100% (20/20), done.
Writing objects: 100% (28/28), 5.74 KiB | 652.00 KiB/s, done.
Total 28 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'master' on GitHub by visiting:
remote:   https://github.com/DoryGR/aulajpa/pull/new/master
remote:
To https://github.com/DoryGR/aulajpa.git
 * [new branch]      master -> master
Branch 'master' set up to track remote branch 'master' from 'origin'.

doryg@Dory-Dell-XPS MINGW64 ~/Documents/ws-spring/aulajpa (master)
$
```



Git

Usando o Git com GitHub

Passo 16

Verifique se a nova versão do projeto está no repositório remoto

 **DoryGR / aulajpa** Private

[Code](#) [Issues](#) [Pull requests](#) [Actions](#) [Projects](#) [Security](#) [Insights](#) [Settings](#)

 master had recent pushes 11 minutes ago [Compare & pull request](#)

 master ▾  2 branches  0 tags [Go to file](#) [Add file ▾](#) [Code ▾](#)

This branch is 1 commit ahead of main. [Pull request](#) [Compare](#)

 DoryGR Versão 1 - Inicial b1b08eb 19 minutes ago 🕒 2 commits		
 .settings	Versão 1 - Inicial	19 minutes ago
 src/main	Versão 1 - Inicial	19 minutes ago
 target/classes/META-INF	Versão 1 - Inicial	19 minutes ago



Arquivo .gitignore

Contém a lista dos tipos de arquivos do projeto que devem ser ignorados e não serão controlados pelo Git

```
# Compiled class file
*.class
```

```
# Log file
*.log
```

```
# BlueJ files
*.ctxt
```

```
# Mobile Tools for Java (J2ME)
.mtj.tmp/
```

```
# Package Files #
*.jar
*.war
*.nar
*.ear
*.zip
*.tar.gz
*.rar
```

Git

Como obter um projeto do GitHub?

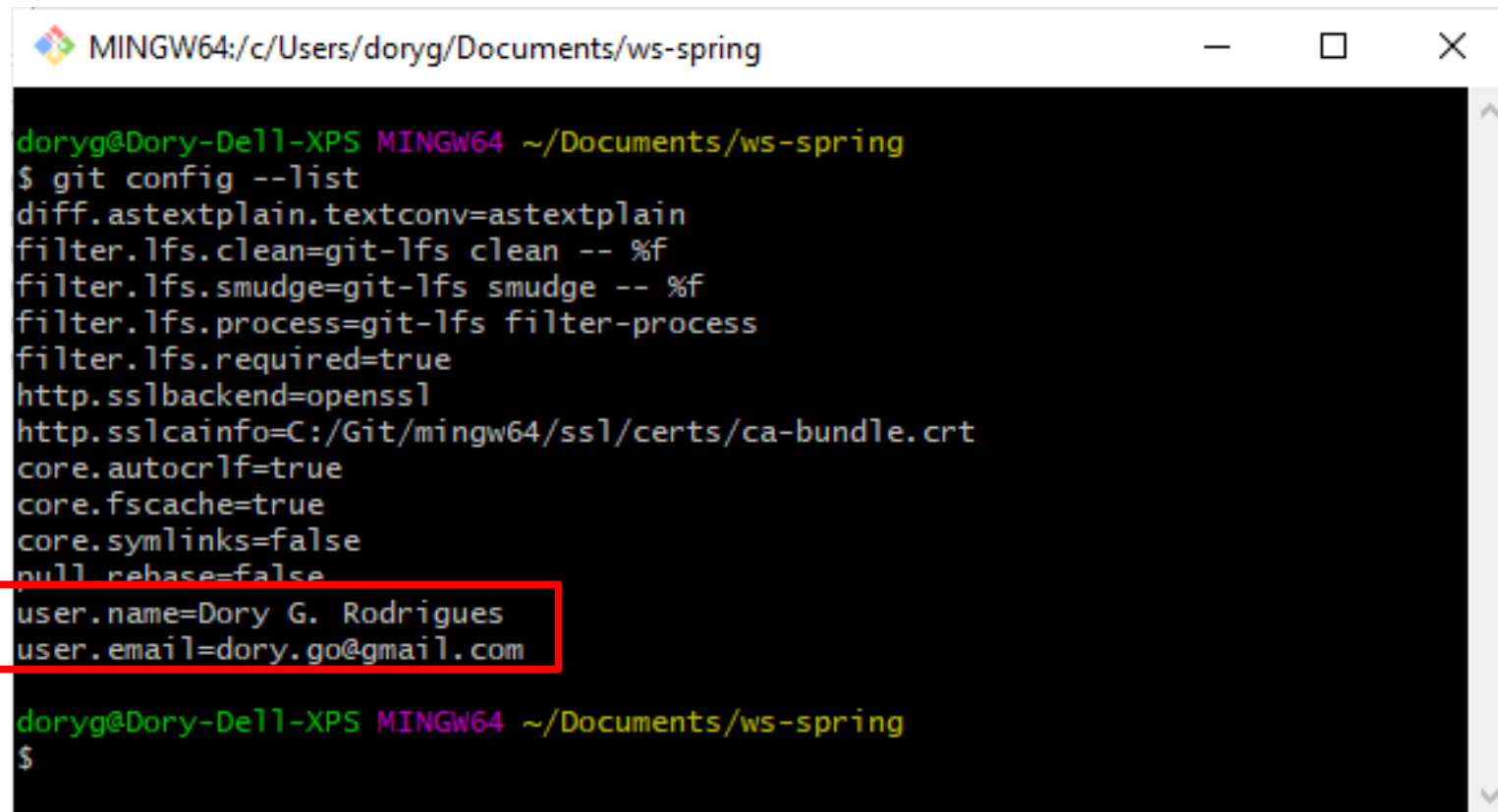
Vamos obter um projeto disponível no repositório remoto.

Passo 1

Verifique se o usuário e e-mail estão configurados no seu repositório local

Comando

`git config --list`



```
MINGW64:/c/Users/doryg/Documents/ws-spring

doryg@Dory-Dell-XPS MINGW64 ~/Documents/ws-spring
$ git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=openssl
http.sslcainfo=C:/Git/mingw64/ssl/certs/ca-bundle.crt
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
user.name=Dory G. Rodrigues
user.email=dory.go@gmail.com

doryg@Dory-Dell-XPS MINGW64 ~/Documents/ws-spring
$
```



Git

Como obter um projeto do GitHub?

Passo 2

Copie o link do projeto contido no repositório remoto do GitHub

The screenshot shows the GitHub interface for the repository 'DoryGR / aulajpa'. The repository name and 'Private' status are highlighted with a red box. Below the repository name, the 'master' branch is selected, also highlighted with a red box. To the right, there are buttons for 'Go to file', 'Add file', and 'Code'. The 'Code' button is clicked, opening a dropdown menu. In this menu, the 'Clone' section is visible, showing options for 'HTTPS', 'SSH', and 'GitHub CLI'. The HTTPS URL 'https://github.com/DoryGR/aulajpa.git' is highlighted with a red box, and a red arrow points to the copy icon next to it. Below the URL, it says 'Use Git or checkout with SVN using the web URL.'

DoryGR / aulajpa Private

Unwatch 1

Code Issues Pull requests Actions Projects Security

master

This branch is 1 commit ahead of main.

DoryGR Versão 1 - Inicial

Clone

HTTPS SSH GitHub CLI

<https://github.com/DoryGR/aulajpa.git>

Use Git or checkout with SVN using the web URL.

Git

Como obter um projeto do GitHub?

Passo 3

Clone o repositório remoto do GitHub

Comando

```
git clone https://github.com/nome_usuario/nome_projeto.git
```

MINGW64:/c/Users/doryg/Documents/ws-spring

```
doryg@Dory-Dell-XPS MINGW64 ~/Documents/ws-spring
$ git clone https://github.com/DoryGR/aulajpa.git
Cloning into 'aulajpa'...
remote: Enumerating objects: 31, done.
remote: Counting objects: 100% (31/31), done.
remote: Compressing objects: 100% (22/22), done.
remote: Total 31 (delta 0), reused 28 (delta 0), pack-reused 0
Unpacking objects: 100% (31/31), 6.47 KiB | 30.00 KiB/s, done.

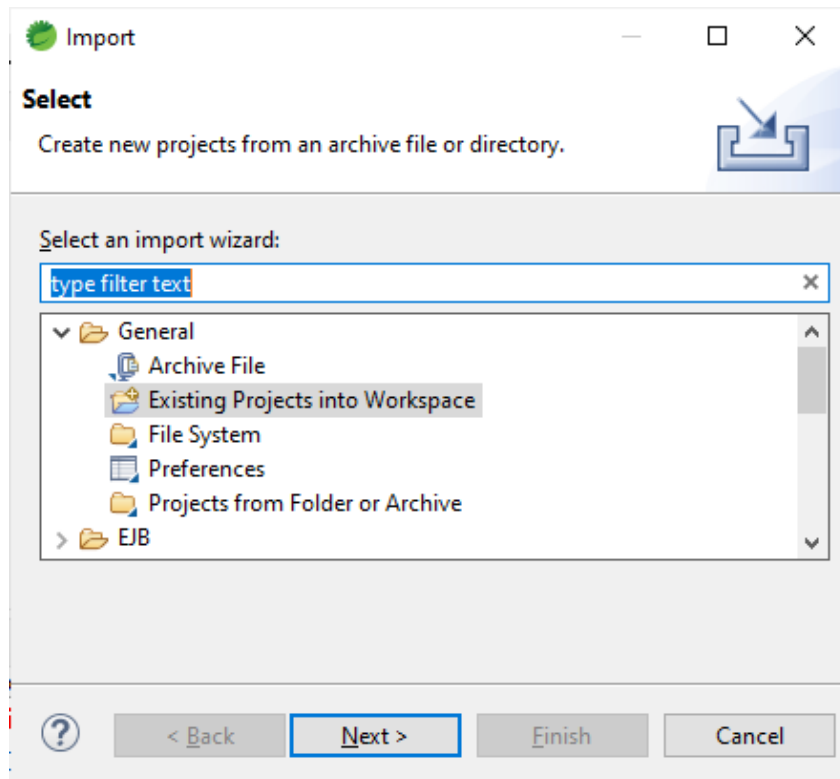
doryg@Dory-Dell-XPS MINGW64 ~/Documents/ws-spring
$
```




Como obter um projeto do GitHub?

Passo 4

Importe o projeto para o framework Java Eclipse ou STS: *Menu File » Import*



a - Selecione » General » Existing Projects into Workspace

b - Clique em Next

c - Selecione a pasta com o repositório Git que contenha o projeto a ser importado

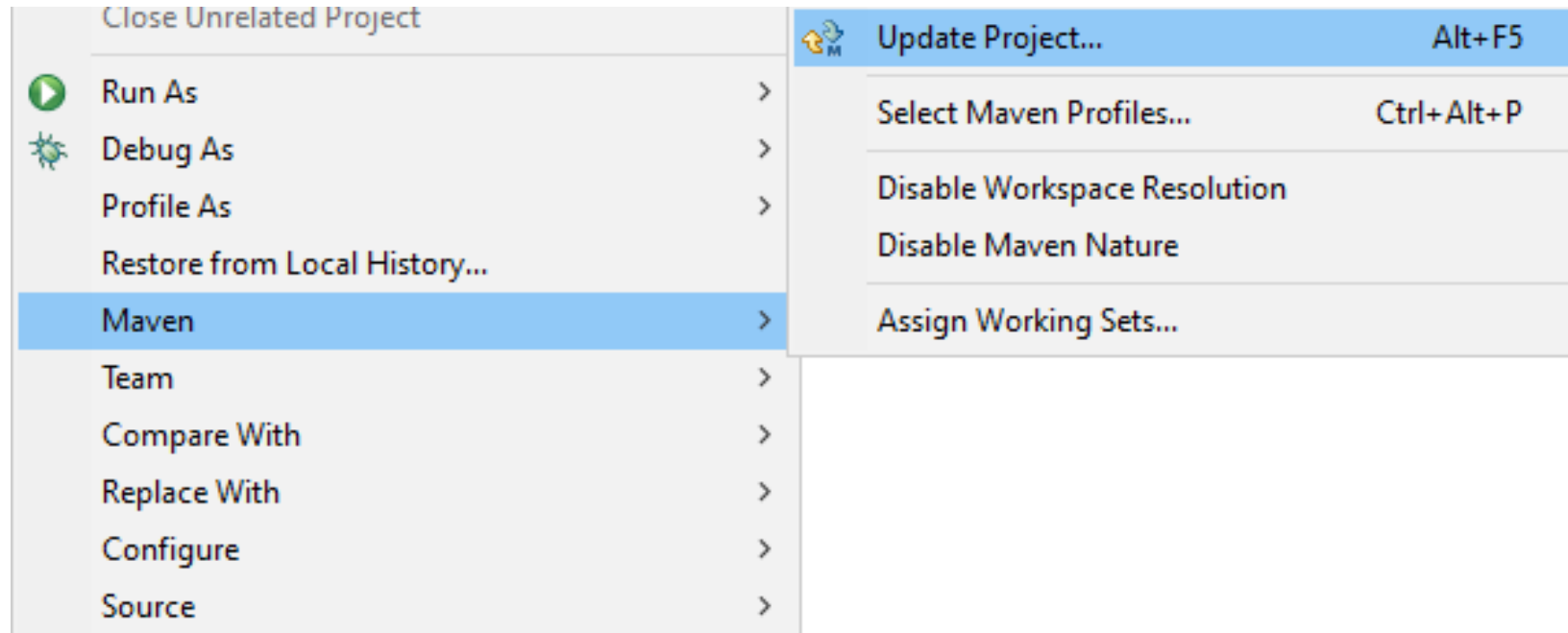
d - Clique em Finish

Git

Como obter um projeto do GitHub?

Passo 5

Se o projeto apresentar algum erro, após a importação, clique com o botão direito no projeto



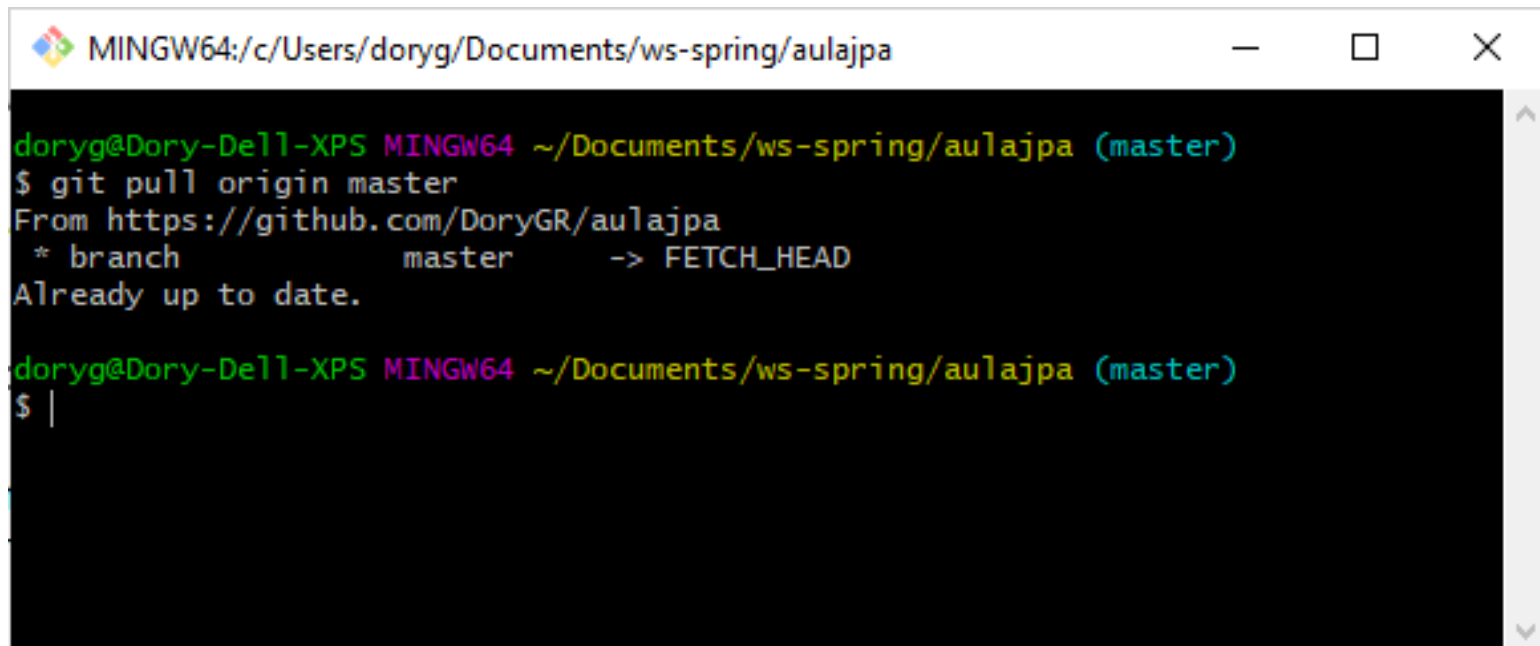
Git

Como atualizar o repositório local?

Antes de iniciar modificações no projeto, devemos atualizar o repositório local com a última versão do repositório remoto:

Comando

`git pull origin master`



```
MINGW64:/c/Users/doryg/Documents/ws-spring/aulajpa
doryg@Dory-De11-XPS MINGW64 ~/Documents/ws-spring/aulajpa (master)
$ git pull origin master
From https://github.com/DoryGR/aulajpa
 * branch          master      -> FETCH_HEAD
Already up to date.

doryg@Dory-De11-XPS MINGW64 ~/Documents/ws-spring/aulajpa (master)
$ |
```

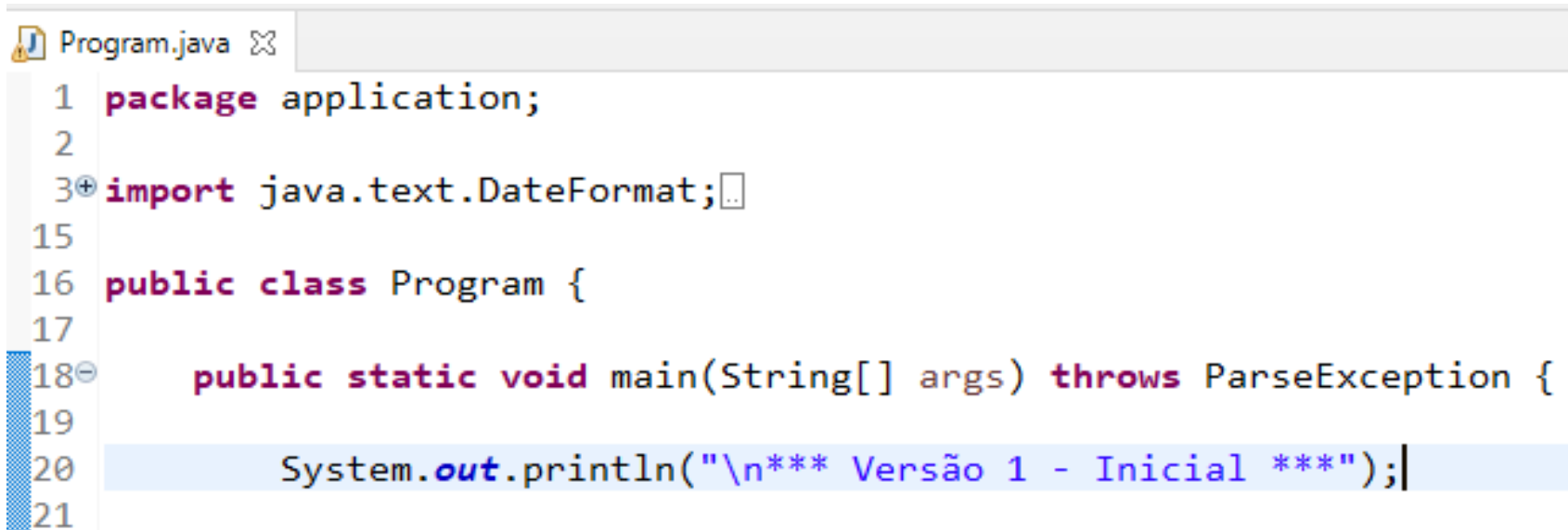


Como atualizar o repositório remoto?

Atualizamos o repositório remoto com as modificações feitas no repositório local da seguinte forma:

Passo 1

Realize as modificações no seu projeto. Como exemplo aterei o arquivo *Program.java* inserindo o texto contido na linha 20.



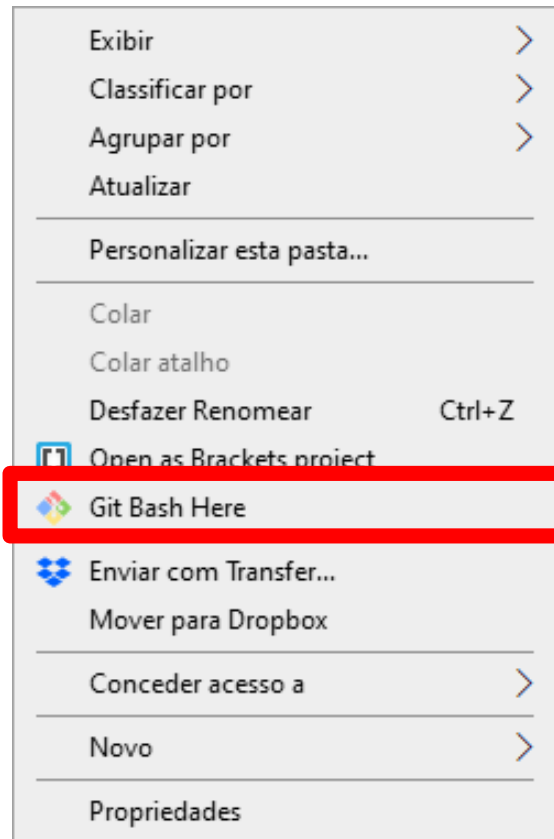
```
Program.java ✕
1 package application;
2
3+ import java.text.DateFormat;
15
16 public class Program {
17
18-     public static void main(String[] args) throws ParseException {
19
20         System.out.println("\n*** Versão 1 - Inicial ***");
21
```

**Git**

Como atualizar o repositório remoto?

Passo 2

No seu computador, dentro da pasta do projeto Java, abra um terminal Git, clicando com o botão direito e escolhendo a opção mostrada abaixo:



Git

Como atualizar o repositório remoto?

Passo 3

Verificar se existem arquivo modificados, não rastreados ou deletados no repositório local

Comando
git status

MINGW64:/c/Users/doryg/Documents/ws-spring/aulajpa

```
doryg@Dory-Dell-XPS MINGW64 ~/Documents/ws-spring/aulajpa (master)
$ git status
On branch master
Your branch is up to date with 'origin/master'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   .classpath
    modified:   src/main/java/application/Program.java
    modified:   target/classes/META-INF/maven/br.com.ifgoias/aulajpa/pom.properties

no changes added to commit (use "git add" and/or "git commit -a")

doryg@Dory-Dell-XPS MINGW64 ~/Documents/ws-spring/aulajpa (master)
$ |
```

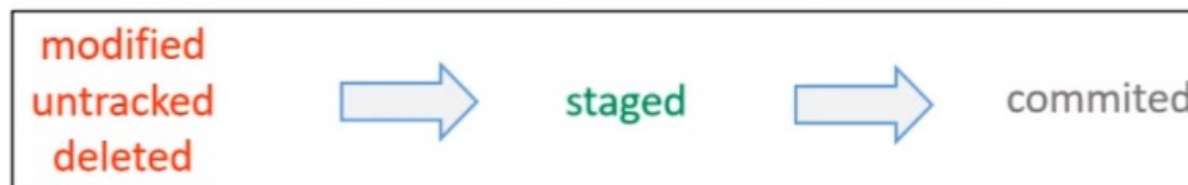


Git

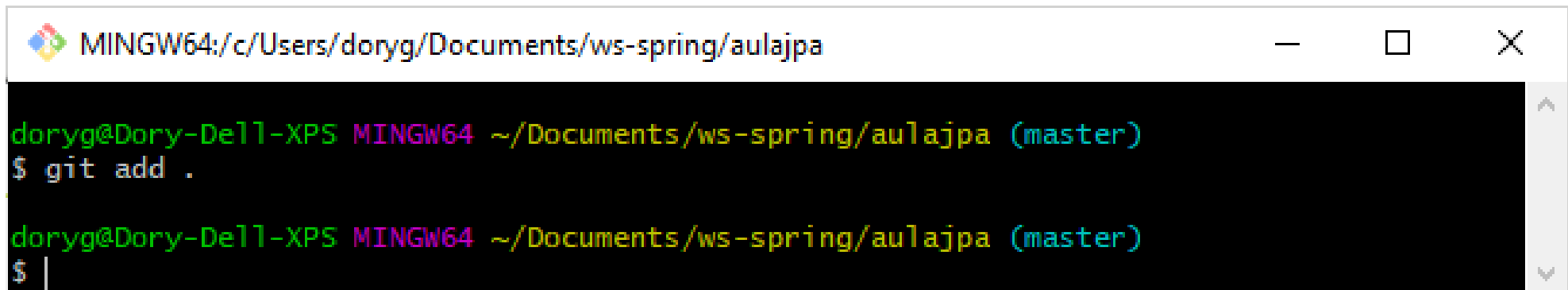
Como atualizar o repositório remoto?

Passo 4

Colocar os arquivos na pasta Stage.



Comando
`git add .`



```
MINGW64:/c/Users/doryg/Documents/ws-spring/aulajpa

doryg@Dory-Dell-XPS MINGW64 ~/Documents/ws-spring/aulajpa (master)
$ git add .

doryg@Dory-Dell-XPS MINGW64 ~/Documents/ws-spring/aulajpa (master)
$ |
```

Git

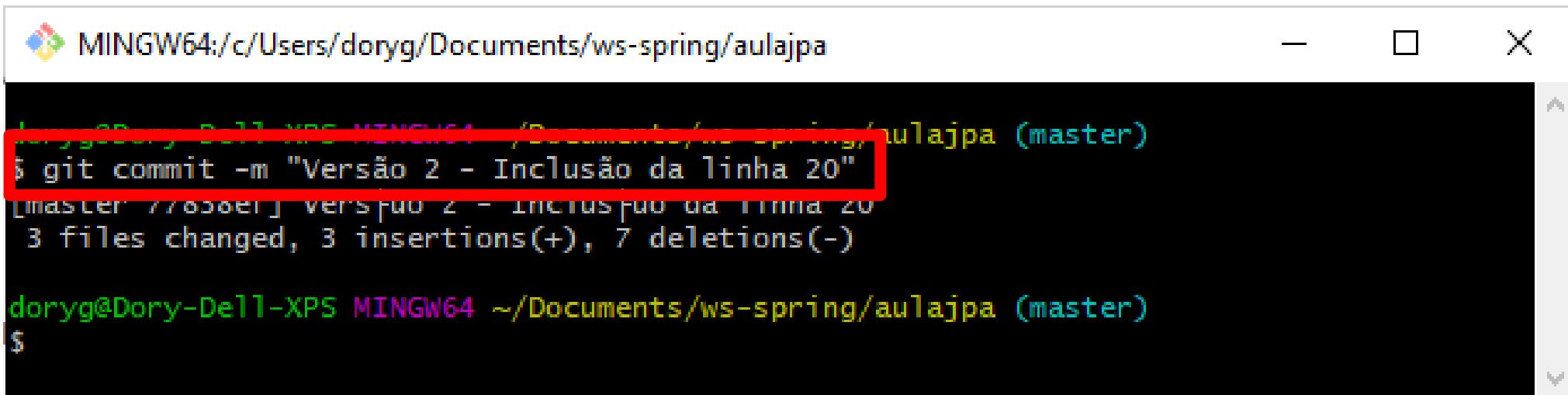
Como atualizar o repositório remoto?

Passo 5

Criar a nova versão do projeto no repositório local.

Comando

`git commit -m "título da versão"`



```
MINGW64:/c/Users/doryg/Documents/ws-spring/aulajpa

doryg@dory-Dell-XPS MINGW64 ~/Documents/ws-spring/aulajpa (master)
$ git commit -m "Versão 2 - Inclusão da linha 20"
[master 77838e1] versão 2 - inclusão da linha 20
3 files changed, 3 insertions(+), 7 deletions(-)

doryg@dory-Dell-XPS MINGW64 ~/Documents/ws-spring/aulajpa (master)
$
```


Git

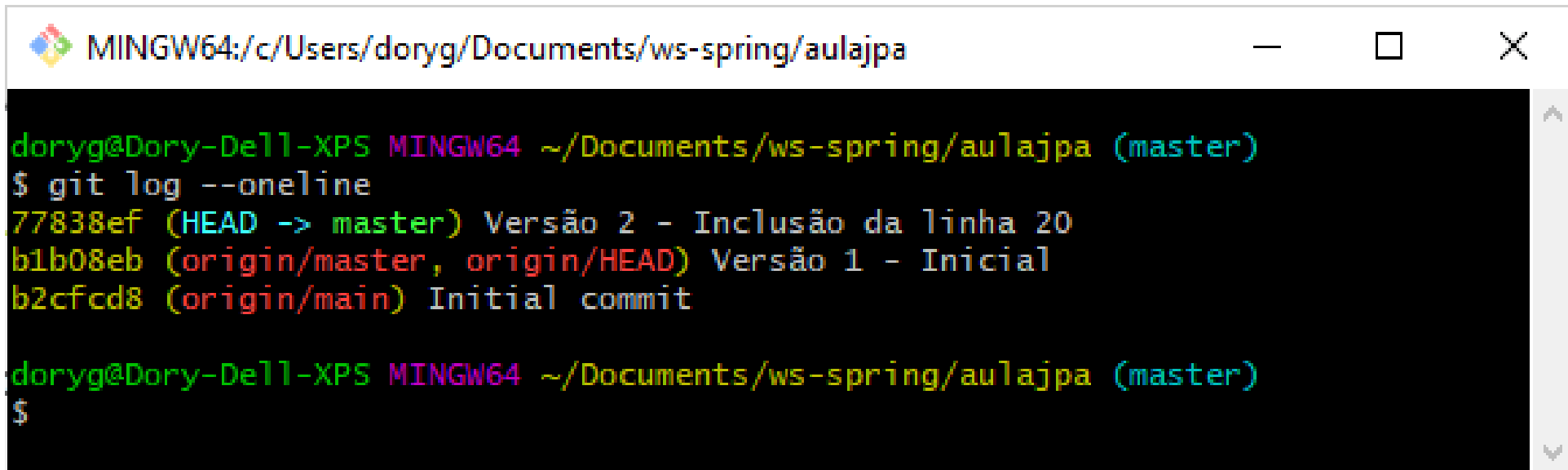
Como atualizar o repositório remoto?

Passo 6

Conferir a nova versão do repositório local

Comando

`git log --online`



```
MINGW64:/c/Users/doryg/Documents/ws-spring/aulajpa

doryg@Dory-Dell-XPS MINGW64 ~/Documents/ws-spring/aulajpa (master)
$ git log --online
77838ef (HEAD -> master) Versão 2 - Inclusão da linha 20
b1b08eb (origin/master, origin/HEAD) Versão 1 - Inicial
b2cfcd8 (origin/main) Initial commit

doryg@Dory-Dell-XPS MINGW64 ~/Documents/ws-spring/aulajpa (master)
$
```

Git

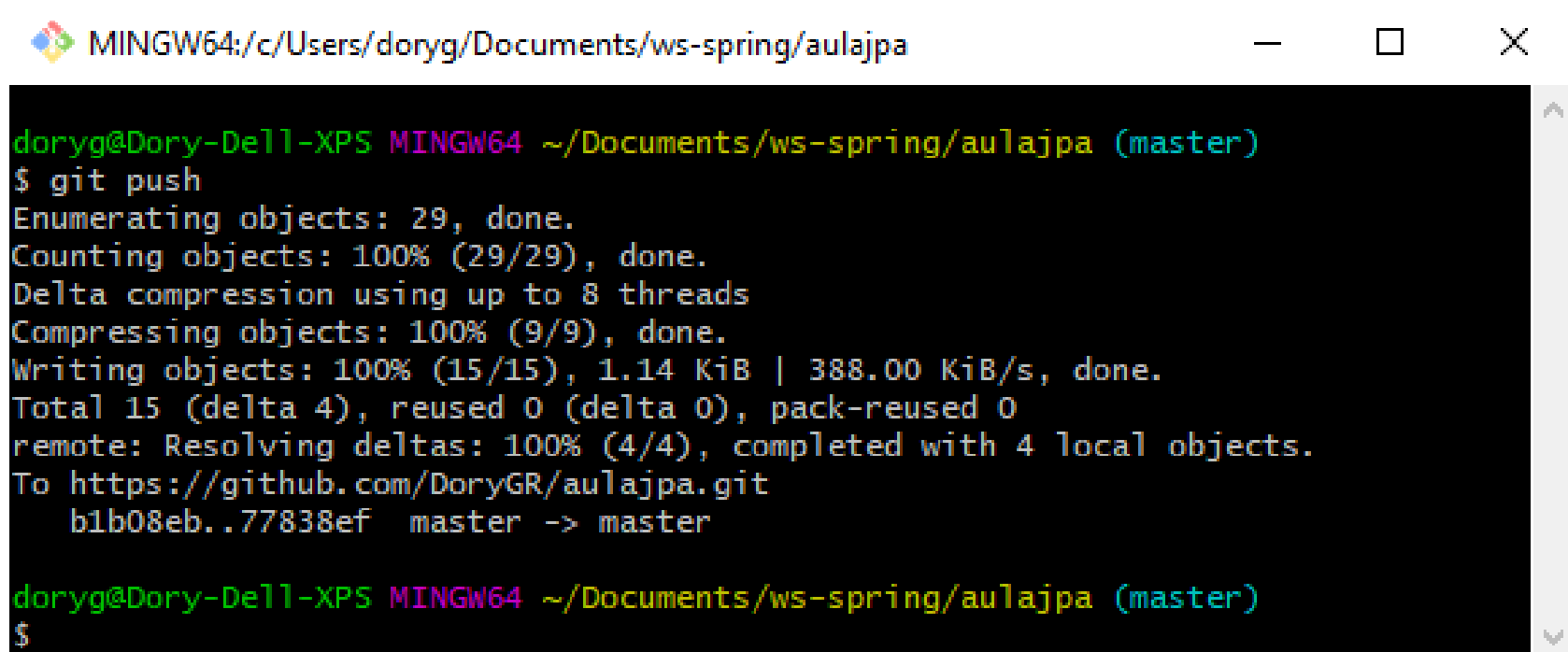
Como atualizar o repositório remoto?

Passo 7

Salvar a nova versão do projeto no repositório remoto (GitHub)

Comando

`git push`



```
MINGW64:/c/Users/doryg/Documents/ws-spring/aulajpa
doryg@Dory-De11-XPS MINGW64 ~/Documents/ws-spring/aulajpa (master)
$ git push
Enumerating objects: 29, done.
Counting objects: 100% (29/29), done.
Delta compression using up to 8 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (15/15), 1.14 KiB | 388.00 KiB/s, done.
Total 15 (delta 4), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (4/4), completed with 4 local objects.
To https://github.com/DoryGR/aulajpa.git
    b1b08eb..77838ef  master -> master
doryg@Dory-De11-XPS MINGW64 ~/Documents/ws-spring/aulajpa (master)
$
```



Git

Como atualizar o repositório remoto?

Passo 8

Verificar o commit no repositório remoto (GitHub)

The screenshot shows the GitHub interface for the repository 'DoryGR / aulajpa'. The repository is marked as 'Private'. The navigation bar includes links for Code, Issues, Pull requests, Actions, Projects, Security, and Insights. Below the navigation bar, there are buttons for 'master' (selected), '1 branch', and '0 tags', along with 'Go to file', 'Add file', and a green 'Code' button. The main content area displays a list of files and folders. A red bracket on the left side of the file list highlights the first four items: '.settings', 'src/main', 'target/classes/META-INF', and '.classpath'. These four items are all associated with the commit 'Versão 2 - Inclusão da linha 20' (77838ef, 23 minutes ago). The other items, '.gitignore', '.project', and 'pom.xml', are associated with 'Versão 1 - Inicial' (7 hours ago).

File/Folder	Commit	Time
.settings	Versão 1 - Inicial	7 hours ago
src/main	Versão 2 - Inclusão da linha 20	23 minutes ago
target/classes/META-INF	Versão 2 - Inclusão da linha 20	23 minutes ago
.classpath	Versão 2 - Inclusão da linha 20	23 minutes ago
.gitignore	Initial commit	8 hours ago
.project	Versão 1 - Inicial	7 hours ago
pom.xml	Versão 1 - Inicial	7 hours ago